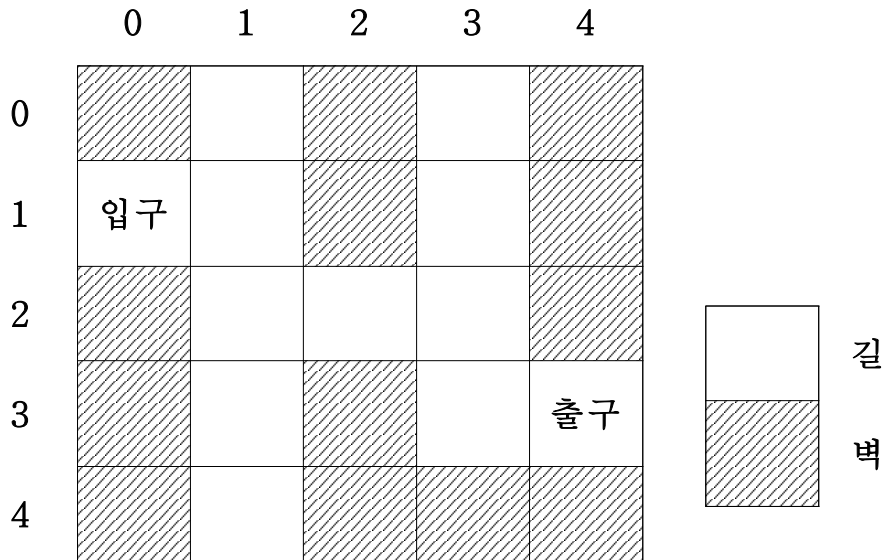


【 문제-1 】 (30점)

아래와 같이 미로가 주어졌을 때 다음의 물음에 답하시오.



(1) 스택과 큐를 이용하는 탐색방식의 개념과 특징을 설명하시오. (4점)

(2) 미로의 입구부터 출구까지의 경로를 찾는 과정에서 스택을 이용하여 다음 <조건>에 따라 출구를 찾을 때까지 <top의 변화과정>의 ①~⑤를 완성하고 이를 구현한 <소스코드>의 ⑥~⑧을 작성하시오. (13점)

<조건>

가) 미로의 이동규칙에 관한 조건은 다음과 같다.

- 우선순위 : 상→하→좌→우
- 한번 지나간 위치는 재이동 금지
- 종료조건 : 출구(3,4) 도착
- 위치의 좌표는 행우선 표시 예) 입구 (1,0), 출구 (3,4)

나) 스택에 관한 조건은 다음과 같다.

- 현재 위치에서 스택에 넣고 이동(입구에서 시작)
- 이동할 수 없는 위치에 도달 시 스택에서 현재 위치 제거 후 이동 (단, 백트래킹은 구현되었다고 가정한다.)
- 스택의 삽입함수와 삭제함수는 push(node)와 pop()이다.

<top의 변화과정>

단계	1	2	3	4	5	6	7	8
top	(1,0)	(1,1)	(0,1)	(2,1)	(3,1)	①	(3,1)	②
단계	9	10	11	12	13	14	15	16
top	(2,2)	③	(1,3)	④	(1,3)	⑤	(3,3)	(3,4)

<소스코드>

```

stack stack;// 위치정보를 담은 스택

void find_path(maps, visited, pos) {
    if(pos == [3, 4])
        return;
    else {
        visited[pos] = true;
        stack. ⑥

        for(i = 0; i < 4; i++) {
            next = pos + offset[i];
            // offset(현재 위치에서 이동할 상하좌우 위치 배열)

            if(is_movable(maps, next))
                ⑦
        }
        stack. ⑧
    }
}

```

(3) 미로의 입구부터 출구까지의 경로를 찾는 과정에서 큐를 이용하여 다음 <조건>에 따라 출구를 찾을 때까지 <큐의 변화과정>의 ①~⑤를 완성하고 이를 구현한 <소스코드>의 ⑥~⑧을 작성하시오. (13점)

<조건>

가) 미로의 이동규칙에 관한 조건은 다음과 같다.

- 우선순위 : 상→하→좌→우
- 한번 지나간 위치는 재이동 금지
- 종료조건 : 출구(3,4) 도착
- 위치의 좌표는 행우선 표시 예) 입구 (1,0), 출구 (3,4)

나) 큐에 관한 조건은 다음과 같다.

- 현재 위치에서 다음 방문 좌표 생성 후 이동(입구에서 시작)
- 다음 위치로 이동이 끝나면 현재 위치를 큐에서 삭제
- 큐의 삽입함수 및 삭제함수는 enqueue(node), dequeue()이다.

<큐의 변화과정>

단계	1	2	3	4	5	6	7	8	9	10	11	12
삭제												
↑	(1,0)	(1,1)	(0,1)	(2,1)	①	(2,2)	(4,1)	③	④	(3,3)	(0,3)	(3,4)
↑			(2,1)		(2,2)	②	(2,3)		(3,3)	⑤	(3,4)	
삽입												

<소스코드>

```
queue queue;    // 위치 정보를 담은 큐
void find_path(maps, start, visited) {

    queue. ⑥

    while( ⑦ ) {
        now = queue.dequeue();
        if(now == end)
            return;

        visited[now] = true;

        for(i = 0; i < 4; i++) {
            next = now + offset[i];
            // offset(현재 위치에서 이동할 상하좌우 위치 배열)
            if(is_movable(maps, next, visited)

                queue. ⑧

            }
        }
    }
}
```

【 문제-2 】 (20점)

공간복잡도에 관한 다음 물음에 답하시오.

(1) Big "O", Big "Ω", Big "Θ" 의 정의를 각각 기술하시오. (6점)

(2) 다음식이 성립함을 설명하시오. (12점)

1) $5\log n = O(\log n)$

2) $4n^2 = \Omega(n^2)$

3) $2n^2 = \Theta(n^2)$

(3) 다음 Big "O" 표기법에 대하여 실행시간이 빠른 순서대로 나열하시오. (2점)

$O(2^n)$, $O(\log n)$, $O(n \log n)$, $O(n^3)$, $O(n^2)$, $O(n)$

【 문제-3 】 (30점)

힙(Heap)에 관한 다음 물음에 답하시오.

(1) 힙(Heap), 최대힙(Max heap), 최소힙(Min heap)의 정의를 각각 기술하시오. (6점)

(2) 다음의 순서로 데이터가 입력될 경우, 최대힙과 최소힙을 구성하고자 한다. 각각에 대하여 데이터가 입력되는 순서마다 힙구조가 변화되는 과정과 최종 결과를 각각 그림으로 나타내시오. (10점)

4, 8, 12, 62, 18, 23, 30, 42

(3) 물음 (2)에서 구성된 최소힙에서 새로운 데이터 9가 삽입되는 과정을 설명하고 그 결과를 그림으로 나타내시오. (7점)

(4) 물음 (2)에서 구성된 최대힙에서 데이터 62가 삭제되는 과정을 설명하고 그 결과를 그림으로 나타내시오. (7점)

【 문제-4 】 (20점)

아래는 이진탐색트리(Binary Search Tree)의 특정 키(노드)를 탐색, 삽입, 삭제하는 의사코드(pseudo code)이다.

<주요 함수 및 조건>을 이용하여 다음의 물음 (1) 탐색(search)의 ①, ②, 물음 (2) 삽입(insert)의 ①~④, 물음 (3) 삭제(remove)의 ①~④를 완성하시오.

<주요 함수 및 조건>

- KEY(node) : node의 키 반환
- LEFT(node) : node의 왼쪽 자식 노드 반환
- RIGHT(node) : node의 오른쪽 자식 노드 반환
- ISLEAF(node) : 단말노드 여부
- DELETE() : 메모리 동적 해제
- 비교는 '=', 치환 · 대입 · 연결은 '←' 로 표시한다.

(1) 탐색(search) (6점)

```
search(root, key)                // root 현재 노드, key 찾을 키
    if root = NULL then
        return NULL;

    if key = KEY(root) then
        return root;

    else if key < KEY(root) then
        ①

    else
        ②
```

(2) 삽입(insert) (6점)

```
insert(root, node)          // root 현재 노드, node 삽입할 노드
    if KEY(node) = KEY(root) then
        return;
```

```
    else if KEY(node) < KEY(root) then
        if LEFT(root) = NULL then
```

①

```
    else
```

②

```
    else
```

```
        if RIGHT(root) = NULL then
```

③

```
        else
```

④

(3) 삭제(remove) (8점)

```
remove(parent, node)    // parent 현재 노드의 부모 노드, node 삭제할 노드
    // case1 : 삭제할 노드가 단말노드인 경우
```

```
    if ISLEAF(node) then
```

```
        if parent = NULL then
```

```
            root ← NULL;
```

```
        else
```

①

```
// case2 : 삭제할 노드가 왼쪽이나 오른쪽 자식 노드만 있는 경우
else if LEFT(node) = NULL or RIGHT(node) = NULL then
    child ← LEFT(node) or RIGHT(node);
```

```
    if node = root then
        root ← child;
    else
```

②

```
// case3 : 삭제할 노드가 두개의 자식노드 모두 있는 경우
else
```

```
    // 첫번째 방안
```

```
        succp ← node;
        succ ← RIGHT(node);
```

③

```
        node ← succ;
```

```
    // 두번째 방안
```

```
        succp ← node;
        succ ← LEFT(node);
```

④

```
        node ← succ;
```

```
DELETE();
```